

University School of Information, Communication and Technology

Guru Gobind Singh Indraprastha University

Dwarka-16'C', Delhi

B. Tech. CSE – 7th Sem

Project Synopsis

on the topic of

Language detection model



Submitted to: Prof. Bharti Suri

Submitted by: Shivam Sah

Enroll No:- 40716403223

1. INTRODUCTION

Language detection is the process of finding which human language is used in a piece of text. It is useful in search engines, chat applications, translation systems, social media, customer support, and many other software systems. This project develops a simple machine learning-based language detection system using text data and a supervised classification model. The project uses a CSV dataset containing 22,000 text samples and their corresponding language labels. The dataset contains 22 languages, with 1,000 samples for each language. The text is converted into numerical features using Count Vectorizer. The data is then divided into training and testing sets. A Multinomial Naive Bayes classifier is trained on the training data. The final model is able to predict the language of new text. Human languages have different words, characters, spellings, and writing patterns. A computer cannot directly use raw text as numerical input for most traditional machine learning algorithms. Therefore, the text must first be converted into numerical features. This project demonstrates that process in a clear and practical way. The system follows a simple pipeline: Load the dataset, inspect and validate the data, convert text into numerical features, split the data, train the classifier, evaluate the model, Prediction In the performed experiment, 14,740 samples were used for training, and 7,260 samples were used for testing. The model achieved a test accuracy of approximately 95.32%. A sample Chinese sentence was also entered as new input, and the model correctly predicted the output as Chinese.

Main Idea of the Project

- **Input:** A sentence or paragraph written in an unknown language.
- **Processing:** Convert the text into numerical word-count features.
- **Classification:** Use Multinomial Naive Bayes to select the most likely language.
- **Output:** The predicted language name.

2. PROBLEM STATEMENT

People communicate using many languages, and software systems often receive text without a language label. Manually identifying the language is slow when large amounts of text are processed. The problem is to build a machine learning system that can automatically classify a text sample into one of the supported languages.

2. OBJECTIVES

The main objectives of this project are:

- To build a simple language detection system using machine learning.
- To use a labelled text dataset containing multiple languages.
- To check the dataset for missing values and understand the class distribution.
- To convert text into numerical features using Count Vectorizer.
- To train a Multinomial Naive Bayes classification model.
- To measure model performance using test accuracy.
- To predict the language of new user-entered text.

3. SCOPE OF THE PROJECT

The project focuses on **text-based language classification**. The current dataset contains **22 languages**, and the model is trained only on those language classes. The system is suitable for demonstration, learning, and small language identification applications. Its performance on languages or writing styles that are not represented in the training data may be limited.

4. SIGNIFICANCE OF THE PROJECT

Automatic language detection can be used as an early step in many **Natural Language Processing (NLP)** systems. For example, a detected language can be passed to:

- A translation system
- A language-specific text processor
- A multilingual search system
- A customer-support system
- A social media analysis system

This project also demonstrates an important machine learning workflow, starting from raw data and ending with a working prediction model.

5. EXPECTED OUTCOME

The expected outcome is a trained classification model that can identify the language of a new text sample. Based on the completed experiment, the model achieved approximately **95.32% accuracy** on the held-out test data.

6. DATASET AND DATA EXPLORATION

The project uses a file named **language.csv**. It contains two columns:

1. **Text**
2. **language**

The **Text** column contains the sentence or paragraph, while the **language** column contains the correct class label.

7. TECHNOLOGIES AND METHODS

1. Python

Python is used to write and run the complete machine learning workflow.

2. NumPy

NumPy is used for array-based data handling.

3. Pandas

Pandas is used to read the CSV file and inspect the dataset.

For example:

```
import pandas as pd  
  
data = pd.read_csv("language.csv")
```

4. Scikit-learn

Scikit-learn is used for:

- Feature extraction
- Data splitting
- Machine learning model training
- Model evaluation
- Prediction

5. Count Vectorizer

Count Vectorizer converts text into numerical **word-count features** that can be understood by the machine learning model.

6. Multinomial Naive Bayes

Multinomial Naive Bayes is the classification algorithm used to predict the language of the input text.

9. METHODOLOGY

Step 1: Load the Dataset

The CSV file is loaded using Pandas.

```
data = pd.read_csv("language.csv")
```

The dataset contains **22,000 rows and 2 columns**.

Step 2: Check the Dataset

Missing values are checked using:

```
data.isnull().sum()
```

The result shows:

Column Missing Values

```
Text      0
```

```
language  0
```

Therefore, there are **no missing values** in the dataset.

Step 3: Check Language Distribution

The number of samples for each language is checked using:

```
data['language'].value_counts()
```

The result shows that **each language contains 1,000 samples**.

Therefore, the dataset has a balanced class distribution.

Step 4: Separate Input and Output

The text is selected as the input variable and language as the target variable.

```
x = np.array(data['Text'])  
y = np.array(data['language'])
```

Here:

- **x = Text data**
- **y = Language labels**

Step 5: Feature Extraction

The text needs to be converted into numerical form.

```
cv = Count Vectorizer()  
x = cv.fit_transform(x)
```

Count Vectorizer creates a vocabulary from the dataset and represents each text sample using numerical word-count features.

Step 6: Train-Test Split

The data is divided into training and testing data.

```
X_train, X_test, y_train, y_test = train_test_split (  
    X, y, test_size=0.33, random_state=42)
```

The dataset is divided approximately as follows:

- **Training Samples:** 14,740
- **Testing Samples:** 7,260

Step 7: Train the Model

A Multinomial Naive Bayes model is created:

```
model = MultinomialNB ()
```

The model is trained using:

```
model.fit(X_train, y_train)
```

The model learns the relationship between the text features and their corresponding language labels.

Step 8: Evaluate the Model

The model is evaluated using:

```
model. Score (X_test, y_test)
```

The obtained score is:

```
0.953168044077135
```

This is approximately:

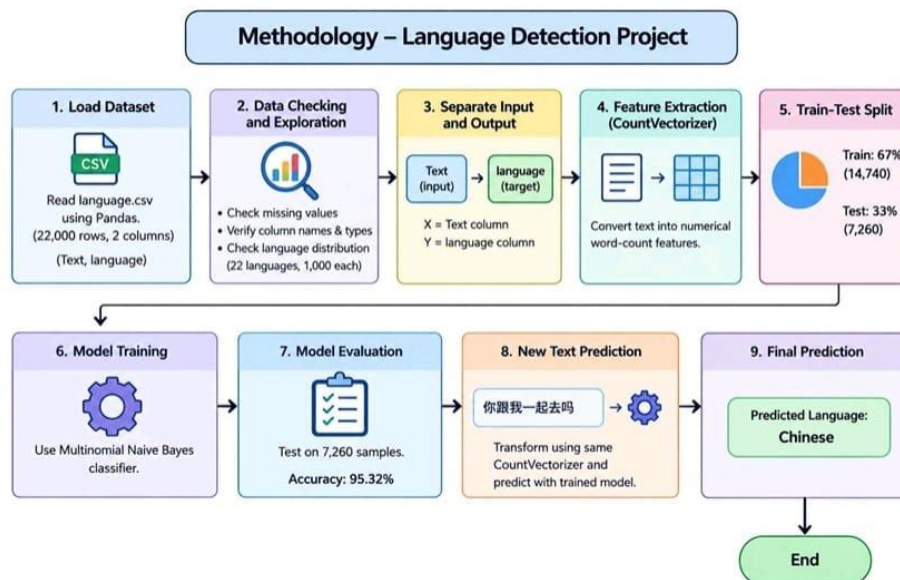
```
95.32% Accuracy
```

Step 9: Predict New Text

A new sentence can be entered by the user:

```
user = input ("Enter a text")  
data = cv transform ([user]). toarray()  
output = model.predict(data)  
print(output)
```

The trained model processes the new text and predicts the most likely language.



8. MODEL TRAINING

The model training process begins after converting the text data into numerical features using **Count Vectorizer**. The dataset is divided into training and testing sets, where: **14,740 samples** are used for training and **7,260 samples** are used for testing.

A **Multinomial Naive Bayes classifier** is then created and trained using the training data. The model learns the relationship between the extracted text features and their corresponding language labels. After training, the model is ready to classify new text into one of the **22 supported languages**.

9. RESULT ANALYSIS

The trained **Multinomial Naive Bayes** model achieved a test accuracy of approximately **95.32%**. This indicates that the model correctly classified most of the text samples in the test dataset. The model was also tested with new user input, such as the Chinese sentence: 你跟我一起去吗 The model correctly predicted the language as **Chinese**. The results show that **Count Vectorizer combined with Multinomial Naive Bayes** can effectively perform language classification on the given dataset.

10. APPLICATIONS

Language detection can be useful in many real-world applications.

1. Multilingual Chat Applications

A chat application can automatically identify the language used by a user.

2. Translation Systems

Language detection can be used as the first step before selecting the appropriate translation model.

3. Customer Support

Customer support systems can identify the language of incoming messages and route them to appropriate language-processing systems.

4. Search Engines

Search engines can use language information to improve search results and processing.

5. Social Media Analysis

Large amounts of social media text can be automatically categorized according to language.

11. CONCLUSION

This project successfully demonstrates **language detection using supervised machine learning**. A dataset containing **22,000 labelled text samples from 22 languages** was loaded and examined. The dataset was checked for missing values and class distribution, and it was found that each language contained **1,000 samples with no missing values**. The text data was converted into numerical features using **CountVectorizer**. The dataset was divided into training and testing sets, with **14,740 samples** used for training and **7,260 samples** used for testing. A **Multinomial Naive Bayes classifier** was trained using the extracted features. The completed model achieved approximately **95.32% accuracy** on the test set. The model was also tested with new user input. For the demonstrated Chinese sentence: 你跟我一起去吗 the model predicted:

Chinese

Overall, the project provides a practical example of how **Natural Language Processing and Machine Learning** can be combined to solve a real-world text classification problem.

12. PROJECT WORKFLOW

The complete workflow of the project can be represented as:

Dataset → Data Checking → Text & Labels → Count Vectorizer → Train/Test Split → Multinomial Naive Bayes → Model Evaluation → New Text → Predicted Language

Final Model Accuracy: 95.32%

13. REFERENCES

1. Scikit-learn Documentation — **Count Vectorizer**.
2. Scikit-learn Documentation — **Multinomial Naive Bayes**.
3. Pandas Documentation — **CSV Data Handling**.
4. NumPy Documentation — **Numerical Arrays**.
5. Machine Learning and Natural Language Processing concepts used in text classification.